

CSCI 4061 - Lab4

10/05/2026



UNIVERSITY OF MINNESOTA

Driven to Discover®

Pipe

- One of the methods for interprocess communication (IPC).
- Used to communicate information between processes in the same family tree
- Has a read end and write end
- Data in pipe follows FIFO order
- Make use of the `pipe` library call

```
#include <unistd.h>
int pipe(int pipefd[2]);
```

- `pipefd[]` is populated with the file descriptors that will be used to read and write.
 - `pipefd[0]` is the read end.
 - `pipefd[1]` is the write end.



Pipe

- Use `read` and `write` functions on the file descriptors. Function semantics slightly different for pipes:
 - `read` is **blocked** if pipe is empty. `write` is **blocked** if pipe is full.
 - If there are no writers, `read` will return end of file once buffer is empty.
 - If there are no readers, `write` will raise a **SIGPIPE**. If this signal is being ignored `write` will fail with the error `EPIPE`.
- Common usage pattern:
 - **Call pipe** in parent process and `fork`. File descriptor table (and therefore `pipefds`) is inherited by child process
 - OS tracks the number of active readers and writers. `close` any end that is not going to be used by the process (important for `read/write` semantics above).
 - `read` and `write` where appropriate.
 - `close` any pipe ends not yet closed.
- Man page on pipes - <https://linux.die.net/man/7/pipe>



Pipe Example

```
char buff[BUFSIZE]
int pipe_fds[2];
pipe(pipe_fds);
pid_t child_pid = fork();
```

} Create pipe

```
if (child_pid == 0) {
    close(pipe_fds[0]);

    dup2(fds[1], STDOUT_FILENO);

    execlp("ls", "ls", "-l", NULL);
} else {
```

} Write data to pipe (child)

```
    close(pipe_fds[1]);
```

```
    read(pipe_fds[0], &buf, BUFF_SIZE);
```

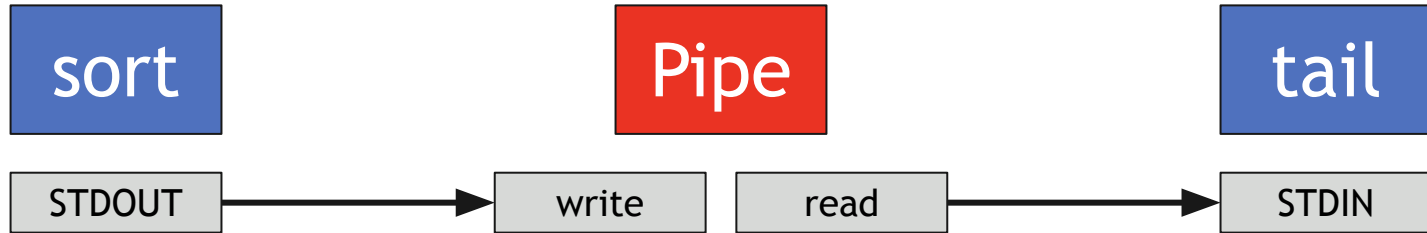
} Read data from pipe (parent)

```
    close(pipe_fds[0]);
```

```
}
```



Lab 4



Lab Reference Sheet

```
// pipefd[0] is read end, pipefd[1] write end
```

```
int pipe(int pipefd[2]);
```

```
int dup2(int oldfd, int newfd); // redirect `newfd` to `oldfd`
```

```
/* Example: */ dup2(my_fd, STDIN_FILENO);
```

```
// ^ redirect standard in to `my_fd`
```

```
// execlp: execute program with arguments as function parameters,  
check PATH env variable
```

```
int execlp(const char *file, const char *arg, ..., NULL);
```

```
/* Example: */ execlp("ls", "ls", "-a", NULL);
```

```
// ^ runs program "ls -a"
```

